INFORME DE CALIDAD EN VERDE

Trabajo Práctico Semana 3: Automatización de Calidad y Blindaje

Asignatura	Metodología de Sistemas II
Carrera	Tecnicatura Universitaria en Programación a Distancia (TUPaD) – UTN
Alumno	Jonathan Nahuel Franco
Fecha	11 de octubre de 2026
Proyecto	calidad-pedidos (Java 17 + Maven): servicio de pedidos de depósito
Repositorio	https://github.com/Yonathannf/calidad-tp3

DICTAMEN: APTO PARA INTEGRACIÓN

El pre-commit hook pasa sus tres controles (formato, linter y pruebas), las 20 pruebas unitarias están en verde con **100 % de cobertura**, el análisis estático no reporta violaciones y las dependencias vulnerables fueron actualizadas. La trazabilidad Issue #1 ↔ Pull Request #2 está establecida.

1. Resumen de resultados

Comparación de las métricas medidas con las herramientas antes y después de la refactorización:

Métrica	Antes	Después
Violaciones de Checkstyle (linter)	**12**	**0**
Violaciones de PMD (complejidad)	**4**	**0**
Bloques duplicados (CPD)	**2**	**0**
Complejidad ciclomática máxima por método	**16 (umbral 10)**	**por debajo del umbral de 10**
Pruebas unitarias	10 pruebas	**20 pruebas, 0 fallos**
Cobertura de instrucciones / ramas (JaCoCo)	no medida	**100 % / 100 %**
Dependencias con versiones vulnerables	**3 (log4j-core, jackson-databind, commons-collections)**	**0 de esas 3**

Nota: el dato de pruebas "antes" corresponde al proyecto base entregado para la auditoría; la cobertura inicial no se midió, por eso solo se informa la final.

2. Punto 1 – Pre-commit hook

2.1 Diseño y configuración

Se implementó un script pre-commit (shell) versionado en el repositorio, en scripts/hooks/pre-commit, y se activó con git config core.hooksPath scripts/hooks. Al ejecutar git commit, Git lo corre automáticamente y ejecuta **de forma secuencial y obligatoria** tres controles:

Paso	Control	Herramienta / comando	Qué verifica
1	Formatter	Spotless (google-java-format) – mvn spotless:check	Que todo el código respete el estilo definido; ante diferencias sugiere `mvn spotless:apply`.
2	Linter	Checkstyle – mvn checkstyle:check	Reglas de diseño: visibilidad, parámetros, complejidad ciclomática (máx. 10), longitud de método (máx. 40) y de línea (máx. 100).
3	Pruebas unitarias	JUnit 5 + JaCoCo – mvn test	Que las 20 pruebas pasen y se genere el reporte de cobertura.

Cada ejecución queda registrada, con fecha y hora, en evidencias/pre-commit.log.

2.2 Qué ocurre en la consola cuando un control detecta un fallo

Ante un fallo el hook imprime el error de la herramienta, luego un mensaje FALLO en el paso N y COMMIT ABORTADO, y termina con **código de salida 1**. Git interpreta ese código como rechazo: el commit no se crea y los pasos siguientes no se ejecutan. Se provocaron y registraron los tres escenarios de fallo reales:

Escenario A – falla el formatter (10/10/2026 22:11:16): Spotless detecta 3 archivos con diferencias de formato.

```
[pre-commit] (1/3) Formatter (Spotless) ...
[INFO] Spotless.Java is keeping 3 files clean - 3 needs changes to be clean
[ERROR] ... The following files had format violations:
[ERROR] Run 'mvn spotless:apply' to fix these violations.
[pre-commit] FALLO en el paso 1: Formatter (Spotless) (codigo de salida 1)
[pre-commit] COMMIT ABORTADO. Corregi el problema y volve a intentar.
```

Escenario B – falla el linter (22:35:37): el formato ya pasa, pero Checkstyle encuentra 12 violaciones.

```
[pre-commit] OK: Formatter (Spotless)
[pre-commit] (2/3) Linter (Checkstyle) ...
[WARN] ...Pedido.java:4:17: La variable 'cliente' debe ser privada ... [VisibilityModifier]
[WARN] ...Pedido.java:13:10: Más de 7 parámetros (encontrado 8). [ParameterNumber]
[WARN] ...PedidoService.java:20:3: La complejidad ciclomática es 12 (máxima permitida es 10).
[WARN] ...PedidoService.java:54:3: La complejidad ciclomática es 16 (máxima permitida es 10).
[ERROR] ... maven-checkstyle-plugin:3.4.0:check ...: You have 12 Checkstyle violations.
[pre-commit] FALLO en el paso 2: Linter (Checkstyle) (codigo de salida 1)
[pre-commit] COMMIT ABORTADO. Corregi el problema y volve a intentar.
```

Escenario C – falla una prueba unitaria (23:52:05): se alteró a propósito un valor esperado; formato y linter pasan, y el paso 3 rechaza el commit.

```
[pre-commit] OK: Formatter (Spotless)
[pre-commit] OK: Linter (Checkstyle)
[pre-commit] (3/3) Pruebas unitarias (JUnit) ...
[ERROR] Tests run: 20, Failures: 1, Errors: 0, Skipped: 0
[ERROR]   PedidoServiceTest.descuentoMayoristaSegunTramo:41 expected: <0.99> but was: <0.2>
[ERROR] BUILD FAILURE
[pre-commit] FALLO en el paso 3: Pruebas unitarias (JUnit) (codigo de salida 1)
[pre-commit] COMMIT ABORTADO. Corregi el problema y volve a intentar.
```

Tras cada fallo se corrigió el problema (o se revirtió el cambio de prueba) y se repitió el commit. El escenario de éxito se muestra en la sección 4.2.

2.3 Por qué esto protege la rama principal

- **Nada defectuoso entra al historial.** Como el commit se aborta, el código sin formato, con deuda de diseño o con pruebas rotas nunca llega a una rama compartida; el problema se corrige en la máquina del desarrollador y no después de integrarlo.
- **Detección temprana y barata.** El control corre en segundos, antes del push y del Pull Request, así la revisión se concentra en la lógica y no en estilo o errores evitables.
- **El orden importa.** Formato → linter → pruebas va de lo más barato a lo más costoso, y el primer fallo corta la cadena para dar feedback inmediato.
- **Evidencia reproducible.** Cada intento queda en pre-commit.log con su fecha, lo que permite auditar qué se rechazó y por qué.

Limitaciones reconocidas. Un hook local puede omitirse con git commit --no-verify, y por eso el script se versiona en el repositorio (core.hooksPath) en lugar de vivir en .git/hooks. Para blindar de verdad la rama principal conviene repetir estos mismos controles en integración continua (por ejemplo GitHub Actions) y exigirlos con reglas de protección de rama.

3. Punto 2 – Auditoría estática y estrategia de refactorización

3.1 Código analizado

Se auditó el módulo ar.edu.utn.tp3.pedidos, que calcula descuentos, costos de envío y totales de pedidos del depósito y controla el stock. El código base se escribió con calidad deliberadamente baja: lógica condicional anidada, tres métodos de total duplicados, campos públicos, un constructor de 8 parámetros y dependencias con versiones antiguas.

3.2 Puntos críticos identificados (métricas "antes")

Herramienta	Hallazgo	Ubicación	Valor / umbral
Checkstyle	VisibilityModifier: 8 campos públicos	Pedido.java (líneas 4 a 11)	debe ser privado
Checkstyle	ParameterNumber: constructor con 8 parámetros	Pedido.java:13	8 (máx. 7)
Checkstyle / PMD	Complejidad ciclomática de calcularDescuento	PedidoService.java:20	12 (máx. 10)
Checkstyle / PMD	Complejidad ciclomática de calcularCostoEnvio	PedidoService.java:54	16 (máx. 10)
Checkstyle	MethodLength de calcularCostoEnvio	PedidoService.java:54	47 líneas (máx. 40)
PMD	Complejidad cognitiva de calcularDescuento	PedidoService.java:20	18 (umbral 15)
PMD	Complejidad cognitiva de calcularCostoEnvio	PedidoService.java:54	31 (umbral 15)
CPD	Duplicación calcularTotalMinorista ↔ calcularTotalMayorista	PedidoService.java:102–122	11 líneas, 63 tokens

Herramienta	Hallazgo	Ubicación	Valor / umbral
CPD	Duplicación entre los tres métodos de total (Minorista, Mayorista, Vip)	PedidoService.java:102–132	11 líneas, 62 tokens

Los dos métodos con más complejidad concentran el mayor riesgo: cuantas más ramas anidadas, más casos hay que probar y más fácil es introducir errores al modificar una tarifa o un porcentaje.

3.3 Dependencias desactualizadas y seguridad

Se usó mvn versions:display-dependency-updates (versions-maven-plugin 2.17.1) con -DallowMajorUpdates=false para obtener la última versión estable sin cambiar de versión mayor. La primera corrida, sin ese filtro, mostraba sugerencias engañosas (una beta y una numeración antigua) que se descartaron.

Dependencia	Antes	Después	Riesgo y criterio
log4j-core	2.14.1	**2.26.1**	La 2.14.1 es vulnerable a Log4Shell (CVE-2021-44228). Se descartó la 3.0.0-beta3 por ser beta y de versión mayor.
jackson-databind	2.9.8	**2.22.3**	La línea 2.9.x tiene vulnerabilidades de deserialización conocidas.
commons-collections	3.2.1	**3.2.2**	La 3.2.1 permite cadenas de deserialización peligrosas, que la 3.2.2 deshabilita por defecto. La "20040616" que sugería Maven no es una versión más nueva real y se descartó.

Reporte "después": versions:display-dependency-updates ya no lista estas tres dependencias; solo quedan actualizaciones menores opcionales de JUnit (5.10.2 → 5.14.4) y del plugin de Checkstyle (10.17.0 → 10.26.1).

3.4 Estrategia de refactorización

Se aplicaron refactorizaciones que no cambian el comportamiento observable y que atacan cada hallazgo:

Problema	Técnica aplicada	Resultado
Campos públicos y constructor de 8 parámetros	Encapsulamiento y "Introduce Parameter Object": Pedido pasa a ser un record compuesto por Cliente, Envio, montoBase y cantidad.	Sin campos públicos; constructor de 4 parámetros.
Alta complejidad en calcularDescuento	Lógica dirigida por datos: el enum TipoCliente guarda su tabla de tramos y devuelve el descuento recorriéndola.	Método simple, nuevos tramos se agregan como datos.
Alta complejidad y longitud en calcularCostoEnvio	El enum Zona guarda tarifas y recargos; el cálculo se divide en métodos chicos (tarifa base, urgente, frágil, volumen).	Método corto; PMD y Checkstyle sin violaciones.
Código duplicado en los tres totales	Consolidar en un único calcularTotal (la diferencia entre tipos de cliente ya está en el descuento).	CPD: 0 duplicaciones.

Problema	Técnica aplicada	Resultado
Condicionales anidados en stock	Cláusulas de guarda en descontarStock y expresión directa en hayStock.	Menos anidamiento, mismos mensajes de error.
Textos mágicos ("CABA", "VIP")	Enums TipoCliente y Zona.	Valores inválidos imposibles; el compilador ayuda.
Cobertura insuficiente	Se pasó de 10 a 20 pruebas: todos los tipos de cliente, todas las zonas y tramos, recargos y las dos excepciones de stock.	100 % de instrucciones y ramas.
Dependencias vulnerables	Actualización a la última versión estable sin cambio de versión mayor (sección 3.3).	Tres dependencias actualizadas.

Como las tres dependencias vulnerables no se usan en el código actual, la recomendación a futuro es eliminarlas del pom.xml si siguen sin uso: lo que no se incluye no puede ser atacado.

4. Punto 3 – Informe de Calidad en Verde

4.1 Enlace trazable entre el Pull Request y el Issue

Issue #1	https://github.com/Yonathannf/calidad-tp3/issues/1
Pull Request #2	https://github.com/Yonathannf/calidad-tp3/pull/2
Vínculo	La descripción del PR comienza con "Closes #1", por lo que al integrarlo GitHub cierra el Issue automáticamente.
Commits	7dbb58a – feat: servicio de pedidos con controles de calidad (commit base, validado por el hook)
	410fbb6 – docs: evidencias de calidad del pre-commit hook (#1) (rama feature/tp3-calidad, PR #2)

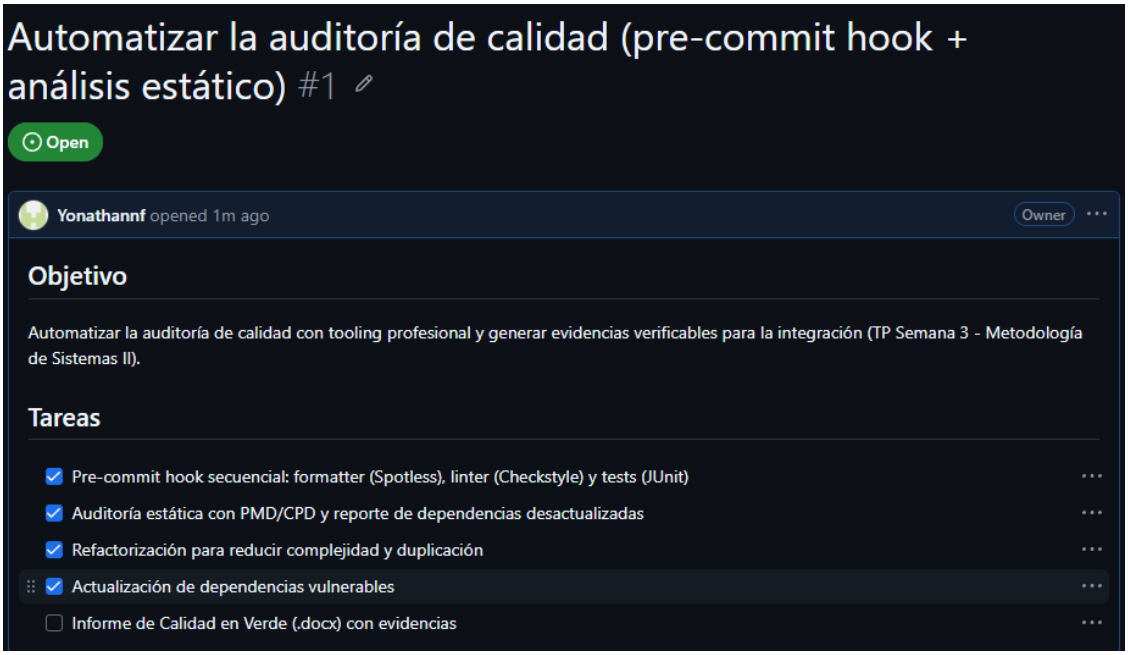


Figura 1. Issue #1 con las tareas y los criterios de aceptación.

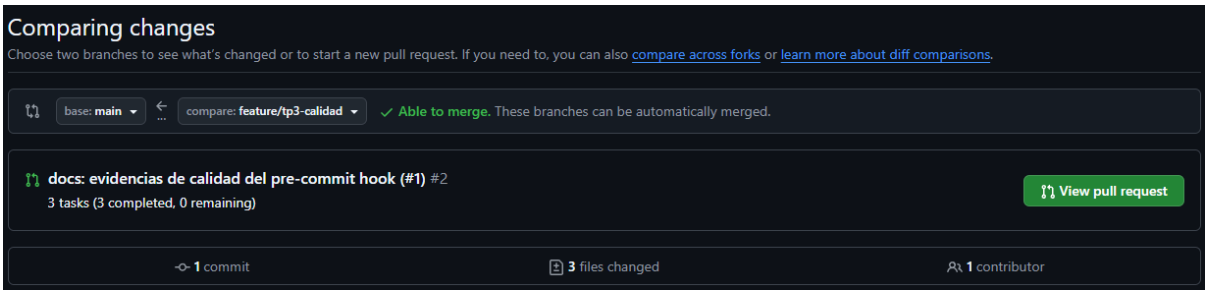


Figura 2. Pull Request #2 con el commit 410fbb6, "Able to merge" y 3 tareas completadas.

4.2 Logs de ejecución del pre-commit hook

Ejecución exitosa registrada al realizar el commit base (10/10/2026 23:46:46) y repetida en el commit del Pull Request (11/10/2026 00:05:17):

```
[pre-commit] (1/3) Formatter (Spotless) ...
[pre-commit] OK: Formatter (Spotless)
[pre-commit] (2/3) Linter (Checkstyle) ...
[INFO] You have 0 Checkstyle violations.
[pre-commit] OK: Linter (Checkstyle)
[pre-commit] (3/3) Pruebas unitarias (JUnit) ...
[INFO] Tests run: 20, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS
[pre-commit] OK: Pruebas unitarias (JUnit)
[pre-commit] TODOS LOS CONTROLES PASARON. Commit permitido.
```

Los logs completos están en el Anexo A. El archivo pre-commit.log es acumulativo: contiene las cinco ejecuciones en orden cronológico (tres con éxito parcial o total y los escenarios de fallo A, B y C).

4.3 Reporte analítico de cobertura de código

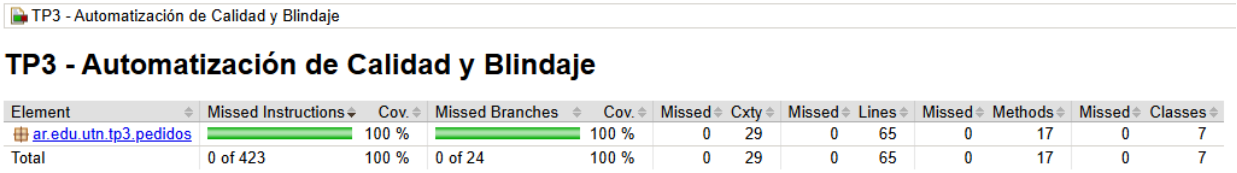


Figura 3. Reporte de JaCoCo 0.8.12 (target/site/jacoco/index.html).

Contador	No cubierto	Cobertura
Instrucciones (423)	0	**100 %**
Ramas (24)	0	**100 %**
Líneas (65)	0	**100 %**
Métodos (17)	0	**100 %**
Clases (7)	0	**100 %**
Complejidad ciclomática (29)	0	**100 %**

4.4 Diagnóstico final de análisis estático

Herramienta	Resultado final	Evidencia
Checkstyle	**0 violaciones**	Log 03 / 04: "You have 0 Checkstyle violations"

Herramienta	Resultado final	Evidencia
PMD (complejidad)	**0 violaciones**	despues-pmd.xml (reporte sin hallazgos)
CPD (duplicación)	**0 bloques duplicados**	despues-cpd.xml (sin elementos duplication)
Dependencias	**3 vulnerables actualizadas**	despues-dependencias.txt
Formato (Spotless)	**sin diferencias**	Log 03: "OK: Formatter (Spotless)"

4.5 Certificación y observaciones

Con las evidencias anteriores se certifica que el desarrollo **cumple los controles de calidad definidos y es apto para su integración** a la rama principal.

Observaciones y trabajo futuro:

- No se ejecutó el escaneo OWASP Dependency-Check; el plugin está configurado en el pom.xml (mvn dependency-check:check, falla con CVSS ≥ 7) y queda como paso siguiente. El riesgo de las dependencias se documentó con el reporte de versiones y las vulnerabilidades públicas conocidas de cada versión.
- El hook es local y puede omitirse con --no-verify: se recomienda replicar los tres controles en integración continua y proteger la rama principal.
- Eliminar del pom.xml las tres dependencias que el código no utiliza.
- Actualizaciones menores opcionales pendientes: JUnit 5.14.4 y Checkstyle 10.26.1.

Anexo A – Inventario de evidencias

Los archivos están en la carpeta evidencias/ del repositorio y fueron generados por las herramientas, no editados.

Archivo	Qué demuestra	Momento
01-fallo-formatter.log	Escenario A: el hook aborta en el paso 1 (Spotless)	10/10/2026 22:11
02-fallo-linter.log	Escenario B: aborta en el paso 2 (12 violaciones de Checkstyle). Incluye el escenario A.	10/10/2026 22:35
03-exito.log	Ejecución exitosa de los tres controles (commit 7dbb58a). Acumula los anteriores.	10/10/2026 23:46
04-fallo-tests.log	Escenario C: aborta en el paso 3 (1 prueba falla). Acumula los anteriores.	10/10/2026 23:52
pre-commit.log	Registro completo y cronológico de las cinco ejecuciones (incluye el commit 410fbb6).	hasta 11/10/2026 00:05
antes-pmd.xml / antes-cpd.xml	Métricas de complejidad y duplicación del código original	10/10/2026 23:33
despues-pmd.xml / despues-cpd.xml	Mismas métricas tras la refactorización (sin hallazgos)	10/10/2026 23:45
antes-dependencias.txt	Versiones desactualizadas antes de actualizar el pom.xml	10/10/2026 23:38
despues-dependencias.txt	Versiones desactualizadas tras actualizar (solo quedan JUnit y Checkstyle)	10/10/2026 23:45

Anexo B – Script del pre-commit hook

Contenido de scripts/hooks/pre-commit:

```
#!/bin/sh
# ---------------------------------------------------------------------
# Pre-commit hook - TP3 Automatizacion de Calidad y Blindaje
# Ejecuta de forma secuencial y obligatoria:
#   1) Formatter  (Spotless / google-java-format)
#   2) Linter     (Checkstyle)
#   3) Tests      (JUnit + JaCoCo)
# Si un paso falla, el commit se aborta (exit 1) y NO se ejecutan los
# pasos siguientes.
# ---------------------------------------------------------------------

LOG_DIR="evidencias"
LOG_FILE="$LOG_DIR/pre-commit.log"
TMP_OUT="$LOG_DIR/.paso.tmp"

mkdir -p "$LOG_DIR"

log() {
    echo "$1" | tee -a "$LOG_FILE"
```

```
}

run_step() {
    numero="$1"
    nombre="$2"
    shift 2
    log ""
    log "[pre-commit] ($numero/3) $nombre ..."
    "$@" > "$TMP_OUT" 2>&1
    estado=$?
    tee -a "$LOG_FILE" < "$TMP_OUT"
    rm -f "$TMP_OUT"
    if [ "$estado" -ne 0 ]; then
        log ""
        log "[pre-commit] FALLO en el paso $numero: $nombre (codigo de salida $estado)"
        log "[pre-commit] COMMIT ABORTADO. Corregi el problema y volve a intentar."
        exit 1
    fi
    log "[pre-commit] OK: $nombre"
}

log "============================================================"
log "[pre-commit] $(date '+%Y-%m-%d %H:%M:%S') - validando antes de commitear"
log "============================================================"

run_step 1 "Formatter (Spotless)" mvn -B -ntp spotless:check
run_step 2 "Linter (Checkstyle)" mvn -B -ntp checkstyle:check
run_step 3 "Pruebas unitarias (JUnit)" mvn -B -ntp test

log ""
log "[pre-commit] TODOS LOS CONTROLES PASARON. Commit permitido."
exit 0
```

Anexo C – Cómo reproducir los controles

```
git config core.hooksPath scripts/hooks              # activar el hook
mvn spotless:check                                   # formatter (spotless:apply lo corrige)
mvn checkstyle:check                                 # linter
mvn test                                             # pruebas + reporte JaCoCo
(target/site/jacoco)
mvn pmd:pmd pmd:cpd                                  # complejidad y duplicación (target/pmd.xml,
cpd.xml)
mvn versions:display-dependency-updates -DallowMajorUpdates=false
mvn dependency-check:check                           # escaneo de CVEs (opcional, lento la
primera vez)
```